TB Burden Reporting Analytics

Seif H. Kungulio

TB Burden Reporting Analytics

Seif H. Kungulio

March 6, 2026

Contents

Introduction **1**

Database Creation **2**

Environment Setup . . . 2
Database Connection . . . 2
Source File Overview . . . 2
Schema Strategy . . . 4
Prerequisites . . . 4
Reset Existing Objects . . . 4
Load the CSV . . . 4
Cleaned Staging Table . . . 6
Data Quality Checks . . . 8
Dimension Tables . . . 8
Country dimension . . . 8
Date dimension . . . 9
Methodology dimension . . . 9
Fact Table . . . 9
Load Dimensions . . . 11
Load Fact Table . . . 12
Performance Indexes . . . 14
Reporting Views . . . 14
Validation Queries . . . 16
Sample Reporting Queries . . . 17

Conclusion **18**

Introduction

To support reporting and analytical exploration of global tuberculosis burden patterns, I designed a reporting database from the `TB_Burden_Country.csv` file using a star schema structure. The dataset stores annual country-level estimates for prevalence, mortality, incidence, HIV-associated TB burden, and case detection rates. Because the source data is already summarized at the country-year level, a dimensional model is a strong fit for reporting, dashboarding, and trend analysis.

The central design decision in this project is the grain of the fact table: **one row per country per year**. Around that grain, I organize descriptive country attributes in `dim_country`, calendar attributes in `dim_date`, and estimate derivation fields in `dim_methodology`. The resulting `fact_tb_burden` table stores the numerical measures required for KPI reporting and comparative analysis. This design makes it easier to answer questions such as which regions have the highest incidence, which countries improved case detection over time, and how mortality and prevalence evolve across years.

Database Creation

Environment Setup

In this setup chunk, I prepare the R environment for the database build. I suppress unnecessary warnings and messages, set the CRAN mirror, and make sure that all required libraries are installed and loaded before any file or database operations begin.

```
knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE)

options(dplyr.print_min = Inf, dplyr.print_max = Inf)
options(repos = c(CRAN = "https://cloud.r-project.org"))

required_pkgs <- c("DBI", "RPostgres", "ini", "readr", "dplyr", "glue")

for (pkg in required_pkgs) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
  library(pkg, character.only = TRUE)
}
```

Database Connection

Here, I read PostgreSQL credentials from a configuration file and establish a safe database connection using `RPostgres`. I also register that connection for later SQL chunks so that all subsequent database commands run against the same session.

```
config <- read.ini("resources/db_config.ini")
db <- config$postgresql

tryCatch({
  con <- dbConnect(
    RPostgres::Postgres(),
    host = db$host,
    dbname = db$database,
    user = db$user,
    password = db$password,
    port = as.integer(db$port)
  )
}, error = function(e) {
  stop("Database connection failed: ", e$message)
})

knitr::opts_chunk$set(connection = con)
```

Source File Overview

Before building the schema, I inspect the uploaded CSV to confirm its structure, verify the number of columns, and ensure the raw headers match the database ingestion design.

```r
tb_raw_preview <- readr::read_csv("resources/TB_Burden_Country.csv", show_col_types = FALSE, n_max = 5)
colnames(tb_raw_preview)
```

```
##  [1] "Country or territory name"
##  [2] "ISO 2-character country/territory code"
##  [3] "ISO 3-character country/territory code"
##  [4] "ISO numeric country/territory code"
##  [5] "Region"
##  [6] "Year"
##  [7] "Estimated total population number"
##  [8] "Estimated prevalence of TB (all forms) per 100 000 population"
##  [9] "Estimated prevalence of TB (all forms) per 100 000 population, low bound"
## [10] "Estimated prevalence of TB (all forms) per 100 000 population, high bound"
## [11] "Estimated prevalence of TB (all forms)"
## [12] "Estimated prevalence of TB (all forms), low bound"
## [13] "Estimated prevalence of TB (all forms), high bound"
## [14] "Method to derive prevalence estimates"
## [15] "Estimated mortality of TB cases (all forms, excluding HIV) per 100 000 population"
## [16] "Estimated mortality of TB cases (all forms, excluding HIV), per 100 000 population, low bound"
## [17] "Estimated mortality of TB cases (all forms, excluding HIV), per 100 000 population, high bound
## [18] "Estimated number of deaths from TB (all forms, excluding HIV)"
## [19] "Estimated number of deaths from TB (all forms, excluding HIV), low bound"
## [20] "Estimated number of deaths from TB (all forms, excluding HIV), high bound"
## [21] "Estimated mortality of TB cases who are HIV-positive, per 100 000 population"
## [22] "Estimated mortality of TB cases who are HIV-positive, per 100 000 population, low bound"
## [23] "Estimated mortality of TB cases who are HIV-positive, per 100 000 population, high bound"
## [24] "Estimated number of deaths from TB in people who are HIV-positive"
## [25] "Estimated number of deaths from TB in people who are HIV-positive, low bound"
## [26] "Estimated number of deaths from TB in people who are HIV-positive, high bound"
## [27] "Method to derive mortality estimates"
## [28] "Estimated incidence (all forms) per 100 000 population"
## [29] "Estimated incidence (all forms) per 100 000 population, low bound"
## [30] "Estimated incidence (all forms) per 100 000 population, high bound"
## [31] "Estimated number of incident cases (all forms)"
## [32] "Estimated number of incident cases (all forms), low bound"
## [33] "Estimated number of incident cases (all forms), high bound"
## [34] "Method to derive incidence estimates"
## [35] "Estimated HIV in incident TB (percent)"
## [36] "Estimated HIV in incident TB (percent), low bound"
## [37] "Estimated HIV in incident TB (percent), high bound"
## [38] "Estimated incidence of TB cases who are HIV-positive per 100 000 population"
## [39] "Estimated incidence of TB cases who are HIV-positive per 100 000 population, low bound"
## [40] "Estimated incidence of TB cases who are HIV-positive per 100 000 population, high bound"
## [41] "Estimated incidence of TB cases who are HIV-positive"
## [42] "Estimated incidence of TB cases who are HIV-positive, low bound"
## [43] "Estimated incidence of TB cases who are HIV-positive, high bound"
## [44] "Method to derive TBHIV estimates"
## [45] "Case detection rate (all forms), percent"
## [46] "Case detection rate (all forms), percent, low bound"
## [47] "Case detection rate (all forms), percent, high bound"
```

```r
dim(tb_raw_preview)
```

```
## [1]  5 47
```

Schema Strategy

The reporting model follows a star schema with the following components:

- `dim_country`: descriptive country and regional attributes
- `dim_date`: year-based reporting calendar
- `dim_methodology`: textual estimate derivation methods
- `fact_tb_burden`: all numeric country-year reporting measures

This design keeps reporting queries simple, supports dashboard filters, and minimizes repeated descriptive text in the fact table.

Prerequisites

I create a dedicated schema named `tb_reporting` to keep all tables, views, and supporting objects isolated and well organized inside PostgreSQL.

```sql
CREATE SCHEMA IF NOT EXISTS tb_reporting;
```

Reset Existing Objects

These chunks drops existing views and tables so the project can be rerun cleanly without conflicts from earlier versions.

```sql
DROP VIEW IF EXISTS tb_reporting.vw_tb_country_year_report;
```

```sql
DROP VIEW IF EXISTS tb_reporting.vw_tb_region_year_summary;
```

```sql
DROP TABLE IF EXISTS tb_reporting.fact_tb_burden CASCADE;
```

```sql
DROP TABLE IF EXISTS tb_reporting.dim_methodology CASCADE;
```

```sql
DROP TABLE IF EXISTS tb_reporting.dim_date CASCADE;
```

```sql
DROP TABLE IF EXISTS tb_reporting.dim_country CASCADE;
```

```sql
DROP TABLE IF EXISTS tb_reporting.stg_tb_burden_country_clean CASCADE;
```

```sql
DROP TABLE IF EXISTS tb_reporting.raw_tb_burden_country CASCADE;
```

Load the CSV

I use an R chunk to load the uploaded CSV file into PostgreSQL. This approach is portable across environments and avoids relying on server-side `COPY` permissions.

```r
tb_raw <- readr::read_csv("resources/TB_Burden_Country.csv", show_col_types = FALSE)

names(tb_raw) <- c(
  "country_name_raw",
  "iso2_code_raw",
  "iso3_code_raw",
  "iso_numeric_code_raw",
  "region_raw",
  "year_raw",
  "estimated_total_population_raw",
  "prevalence_per_100k_raw",
  "prevalence_per_100k_low_raw",
  "prevalence_per_100k_high_raw",
  "prevalence_total_raw",
  "prevalence_total_low_raw",
  "prevalence_total_high_raw",
  "prevalence_method_raw",
  "mortality_excl_hiv_per_100k_raw",
  "mortality_excl_hiv_per_100k_low_raw",
  "mortality_excl_hiv_per_100k_high_raw",
  "deaths_excl_hiv_raw",
  "deaths_excl_hiv_low_raw",
  "deaths_excl_hiv_high_raw",
  "mortality_hiv_pos_per_100k_raw",
  "mortality_hiv_pos_per_100k_low_raw",
  "mortality_hiv_pos_per_100k_high_raw",
  "deaths_hiv_pos_raw",
  "deaths_hiv_pos_low_raw",
  "deaths_hiv_pos_high_raw",
  "mortality_method_raw",
  "incidence_per_100k_raw",
  "incidence_per_100k_low_raw",
  "incidence_per_100k_high_raw",
  "incident_cases_raw",
  "incident_cases_low_raw",
  "incident_cases_high_raw",
  "incidence_method_raw",
  "hiv_in_incident_tb_percent_raw",
  "hiv_in_incident_tb_percent_low_raw",
  "hiv_in_incident_tb_percent_high_raw",
  "incidence_hiv_pos_per_100k_raw",
  "incidence_hiv_pos_per_100k_low_raw",
  "incidence_hiv_pos_per_100k_high_raw",
  "incidence_hiv_pos_raw",
  "incidence_hiv_pos_low_raw",
  "incidence_hiv_pos_high_raw",
  "tbhiv_method_raw",
  "case_detection_rate_percent_raw",
  "case_detection_rate_percent_low_raw",
  "case_detection_rate_percent_high_raw"
)

DBI::dbWriteTable(
```

```
  con,
  name = DBI::Id(schema = "tb_reporting", table = "raw_tb_burden_country"),
  value = tb_raw,
  overwrite = TRUE,
  row.names = FALSE
)
```

Cleaned Staging Table

Next, I build a cleaned staging table that renames the verbose raw headers into SQL-friendly field names, trims whitespace, converts empty strings to `NULL`, and casts values to their proper numeric and date-compatible data types.

```
CREATE TABLE tb_reporting.stg_tb_burden_country_clean AS
SELECT
    NULLIF(TRIM(country_name_raw), '') AS country_name,
    NULLIF(TRIM(iso2_code_raw), '') AS iso2_code,
    NULLIF(TRIM(iso3_code_raw), '') AS iso3_code,
    NULLIF(TRIM(iso_numeric_code_raw::TEXT), '')::INT
    AS iso_numeric_code,
    NULLIF(TRIM(region_raw), '') AS region,
    NULLIF(TRIM(year_raw::TEXT), '')::INT AS year,
    NULLIF(TRIM(estimated_total_population_raw::TEXT), '')::BIGINT
    AS estimated_total_population,

    NULLIF(TRIM(prevalence_per_100k_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_per_100k,
    NULLIF(TRIM(prevalence_per_100k_low_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_per_100k_low,
    NULLIF(TRIM(prevalence_per_100k_high_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_per_100k_high,
    NULLIF(TRIM(prevalence_total_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_total,
    NULLIF(TRIM(prevalence_total_low_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_total_low,
    NULLIF(TRIM(prevalence_total_high_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_total_high,
    NULLIF(TRIM(prevalence_method_raw), '') AS prevalence_method,

    NULLIF(TRIM(mortality_excl_hiv_per_100k_raw::TEXT), '')::NUMERIC(18,2)
    AS mortality_excl_hiv_per_100k,
    NULLIF(TRIM(mortality_excl_hiv_per_100k_low_raw::TEXT), '')::NUMERIC(18,2)
    AS mortality_excl_hiv_per_100k_low,
    NULLIF(TRIM(mortality_excl_hiv_per_100k_high_raw::TEXT), '')::NUMERIC(18,2)
    AS mortality_excl_hiv_per_100k_high,
    NULLIF(TRIM(deaths_excl_hiv_raw::TEXT), '')::NUMERIC(18,2)
    AS deaths_excl_hiv,
    NULLIF(TRIM(deaths_excl_hiv_low_raw::TEXT), '')::NUMERIC(18,2)
    AS deaths_excl_hiv_low,
    NULLIF(TRIM(deaths_excl_hiv_high_raw::TEXT), '')::NUMERIC(18,2)
    AS deaths_excl_hiv_high,

    NULLIF(TRIM(mortality_hiv_pos_per_100k_raw::TEXT), '')::NUMERIC(18,2)
```

```
    AS mortality_hiv_pos_per_100k,
    NULLIF(TRIM(mortality_hiv_pos_per_100k_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS mortality_hiv_pos_per_100k_low,
    NULLIF(TRIM(mortality_hiv_pos_per_100k_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS mortality_hiv_pos_per_100k_high,
    NULLIF(TRIM(deaths_hiv_pos_raw::TEXT), ''))::NUMERIC(18,2)
    AS deaths_hiv_pos,
    NULLIF(TRIM(deaths_hiv_pos_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS deaths_hiv_pos_low,
    NULLIF(TRIM(deaths_hiv_pos_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS deaths_hiv_pos_high,

    NULLIF(TRIM(mortality_method_raw), '') AS mortality_method,

    NULLIF(TRIM(incidence_per_100k_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_per_100k,
    NULLIF(TRIM(incidence_per_100k_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_per_100k_low,
    NULLIF(TRIM(incidence_per_100k_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_per_100k_high,
    NULLIF(TRIM(incident_cases_raw::TEXT), ''))::NUMERIC(18,2)
    AS incident_cases,
    NULLIF(TRIM(incident_cases_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS incident_cases_low,
    NULLIF(TRIM(incident_cases_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS incident_cases_high,

    NULLIF(TRIM(incidence_method_raw), '') AS incidence_method,

    NULLIF(TRIM(hiv_in_incident_tb_percent_raw::TEXT), ''))::NUMERIC(18,2)
    AS hiv_in_incident_tb_percent,
    NULLIF(TRIM(hiv_in_incident_tb_percent_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS hiv_in_incident_tb_percent_low,
    NULLIF(TRIM(hiv_in_incident_tb_percent_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS hiv_in_incident_tb_percent_high,

    NULLIF(TRIM(incidence_hiv_pos_per_100k_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_hiv_pos_per_100k,
    NULLIF(TRIM(incidence_hiv_pos_per_100k_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_hiv_pos_per_100k_low,
    NULLIF(TRIM(incidence_hiv_pos_per_100k_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_hiv_pos_per_100k_high,
    NULLIF(TRIM(incidence_hiv_pos_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_hiv_pos,
    NULLIF(TRIM(incidence_hiv_pos_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_hiv_pos_low,
    NULLIF(TRIM(incidence_hiv_pos_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_hiv_pos_high,

    NULLIF(TRIM(tbhiv_method_raw::TEXT), '') AS tbhiv_method,

    NULLIF(TRIM(case_detection_rate_percent_raw::TEXT), ''))::NUMERIC(18,2)
    AS case_detection_rate_percent,
```

```
    NULLIF(TRIM(case_detection_rate_percent_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS case_detection_rate_percent_low,
    NULLIF(TRIM(case_detection_rate_percent_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS case_detection_rate_percent_high
FROM tb_reporting.raw_tb_burden_country;
```

Data Quality Checks

I validate the business grain and overall completeness of the staging layer before creating the dimensional model.

```
-- Row count in stage
SELECT COUNT(*) AS stage_row_count
FROM tb_reporting.stg_tb_burden_country_clean;
```

Table 1: 1 records

stage_row_count
5120

```
-- Duplicate business grain check
SELECT country_name, year, COUNT(*) AS duplicate_count
FROM tb_reporting.stg_tb_burden_country_clean
GROUP BY country_name, year
HAVING COUNT(*) > 1;
```

Table 2: 0 records

country_name	year	duplicate_count

```
-- Missing required grain fields
SELECT *
FROM tb_reporting.stg_tb_burden_country_clean
WHERE country_name IS NULL OR year IS NULL;
```

Table 3: 0 records

| country_name | iso2 | iso3 | who_region | g_whoregion | e_pop_num | e_inc_num | e_inc_100k | e_inc_100k_lo | e_inc_100k_hi | e_mort_exc_tbhiv_num | e_mort_exc_tbhiv_100k | e_mort_exc_tbhiv_100k_lo | e_mort_exc_tbhiv_100k_hi | e_mortinc_num | e_mortinc_100k | e_mortinc_100k_lo | e_mortinc_100k_hi | e_prev_num | e_prev_100k | e_prev_100k_lo | e_prev_100k_hi | c_cdr | c_cdr_lo | c_cdr_hi | year | c_newinc_100k | c_newinc_100k_lo | c_newinc_100k_hi | c_cdr_prev | c_cdr_prev_lo | c_cdr_prev_hi | c_cdr_prev_ref | c_cdr_prev_ref_lo | c_cdr_prev_ref_hi | c_cdr_prev_ref_ref | c_cdr_prev_ref_ref_lo | c_cdr_prev_ref_ref_hi | c_cdr_prev_ref_ref_ref | c_cdr_prev_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_lo | c_cdr_prev_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_ref_hi | c_cdr_prev_ref

```sql
CREATE TABLE tb_reporting.dim_country (
    country_key BIGSERIAL PRIMARY KEY,
    country_name VARCHAR(150) NOT NULL,
    iso2_code VARCHAR(2),
    iso3_code VARCHAR(3),
    iso_numeric_code INT,
    region VARCHAR(100),
    CONSTRAINT uq_dim_country UNIQUE (country_name, iso3_code)
);
```

Date dimension

The `dim_date` table stores the reporting year and its decade grouping. Since the dataset is annual, the year itself serves as a natural reporting key.

```sql
CREATE TABLE tb_reporting.dim_date (
    date_key INT PRIMARY KEY,
    year INT NOT NULL,
    decade INT NOT NULL,
    year_label VARCHAR(10) NOT NULL,
    CONSTRAINT uq_dim_date UNIQUE (year)
);
```

Methodology dimension

I create `dim_methodology` to hold estimate derivation methods for prevalence, mortality, incidence, and TB-HIV calculations. This avoids repeating the same text strings across the fact table and makes methodology-based filtering easier.

```sql
CREATE TABLE tb_reporting.dim_methodology (
    methodology_key BIGSERIAL PRIMARY KEY,
    prevalence_method VARCHAR(255),
    mortality_method VARCHAR(255),
    incidence_method VARCHAR(255),
    tbhiv_method VARCHAR(255),
    CONSTRAINT uq_dim_methodology UNIQUE (
        prevalence_method,
        mortality_method,
        incidence_method,
        tbhiv_method
    )
);
```

Fact Table

The fact table stores all numeric reporting measures at the **country-year** grain. It references the country, date, and methodology dimensions and acts as the main table for analytical reporting.

```sql
CREATE TABLE tb_reporting.fact_tb_burden (
    tb_burden_key BIGSERIAL PRIMARY KEY,
```

```sql
    country_key BIGINT NOT NULL,
    date_key INT NOT NULL,
    methodology_key BIGINT,

    estimated_total_population BIGINT,

    prevalence_per_100k NUMERIC(18,2),
    prevalence_per_100k_low NUMERIC(18,2),
    prevalence_per_100k_high NUMERIC(18,2),
    prevalence_total NUMERIC(18,2),
    prevalence_total_low NUMERIC(18,2),
    prevalence_total_high NUMERIC(18,2),

    mortality_excl_hiv_per_100k NUMERIC(18,2),
    mortality_excl_hiv_per_100k_low NUMERIC(18,2),
    mortality_excl_hiv_per_100k_high NUMERIC(18,2),
    deaths_excl_hiv NUMERIC(18,2),
    deaths_excl_hiv_low NUMERIC(18,2),
    deaths_excl_hiv_high NUMERIC(18,2),

    mortality_hiv_pos_per_100k NUMERIC(18,2),
    mortality_hiv_pos_per_100k_low NUMERIC(18,2),
    mortality_hiv_pos_per_100k_high NUMERIC(18,2),
    deaths_hiv_pos NUMERIC(18,2),
    deaths_hiv_pos_low NUMERIC(18,2),
    deaths_hiv_pos_high NUMERIC(18,2),

    incidence_per_100k NUMERIC(18,2),
    incidence_per_100k_low NUMERIC(18,2),
    incidence_per_100k_high NUMERIC(18,2),
    incident_cases NUMERIC(18,2),
    incident_cases_low NUMERIC(18,2),
    incident_cases_high NUMERIC(18,2),

    hiv_in_incident_tb_percent NUMERIC(18,2),
    hiv_in_incident_tb_percent_low NUMERIC(18,2),
    hiv_in_incident_tb_percent_high NUMERIC(18,2),

    incidence_hiv_pos_per_100k NUMERIC(18,2),
    incidence_hiv_pos_per_100k_low NUMERIC(18,2),
    incidence_hiv_pos_per_100k_high NUMERIC(18,2),
    incidence_hiv_pos NUMERIC(18,2),
    incidence_hiv_pos_low NUMERIC(18,2),
    incidence_hiv_pos_high NUMERIC(18,2),

    case_detection_rate_percent NUMERIC(18,2),
    case_detection_rate_percent_low NUMERIC(18,2),
    case_detection_rate_percent_high NUMERIC(18,2),

    CONSTRAINT fk_fact_tb_country
        FOREIGN KEY (country_key)
        REFERENCES tb_reporting.dim_country(country_key),
    CONSTRAINT fk_fact_tb_date
```

```
        FOREIGN KEY (date_key)
        REFERENCES tb_reporting.dim_date(date_key),
    CONSTRAINT fk_fact_tb_methodology
        FOREIGN KEY (methodology_key)
        REFERENCES tb_reporting.dim_methodology(methodology_key),
    CONSTRAINT uq_fact_tb_burden UNIQUE (country_key, date_key)
);
```

Load Dimensions

I populate each dimension from the cleaned staging table using distinct records only. This ensures the model remains normalized and avoids unnecessary duplicates.

```
INSERT INTO tb_reporting.dim_country (
    country_name,
    iso2_code,
    iso3_code,
    iso_numeric_code,
    region
)
SELECT DISTINCT
    country_name,
    iso2_code,
    iso3_code,
    iso_numeric_code,
    region
FROM tb_reporting.stg_tb_burden_country_clean
WHERE country_name IS NOT NULL
ORDER BY country_name;
```

```
INSERT INTO tb_reporting.dim_date (
    date_key,
    year,
    decade,
    year_label
)
SELECT DISTINCT
    year AS date_key,
    year,
    (year / 10) * 10 AS decade,
    year::VARCHAR(10) AS year_label
FROM tb_reporting.stg_tb_burden_country_clean
WHERE year IS NOT NULL
ORDER BY year;
```

```
INSERT INTO tb_reporting.dim_methodology (
    prevalence_method,
    mortality_method,
    incidence_method,
    tbhiv_method
)
```

```sql
SELECT DISTINCT
    prevalence_method,
    mortality_method,
    incidence_method,
    tbhiv_method
FROM tb_reporting.stg_tb_burden_country_clean
ORDER BY 1,2,3,4;
```

Load Fact Table

This chunk resolves surrogate keys from the dimensions and inserts all numerical measures into the central fact table.

```sql
INSERT INTO tb_reporting.fact_tb_burden (
    country_key,
    date_key,
    methodology_key,
    estimated_total_population,
    prevalence_per_100k,
    prevalence_per_100k_low,
    prevalence_per_100k_high,
    prevalence_total,
    prevalence_total_low,
    prevalence_total_high,
    mortality_excl_hiv_per_100k,
    mortality_excl_hiv_per_100k_low,
    mortality_excl_hiv_per_100k_high,
    deaths_excl_hiv,
    deaths_excl_hiv_low,
    deaths_excl_hiv_high,
    mortality_hiv_pos_per_100k,
    mortality_hiv_pos_per_100k_low,
    mortality_hiv_pos_per_100k_high,
    deaths_hiv_pos,
    deaths_hiv_pos_low,
    deaths_hiv_pos_high,
    incidence_per_100k,
    incidence_per_100k_low,
    incidence_per_100k_high,
    incident_cases,
    incident_cases_low,
    incident_cases_high,
    hiv_in_incident_tb_percent,
    hiv_in_incident_tb_percent_low,
    hiv_in_incident_tb_percent_high,
    incidence_hiv_pos_per_100k,
    incidence_hiv_pos_per_100k_low,
    incidence_hiv_pos_per_100k_high,
    incidence_hiv_pos,
    incidence_hiv_pos_low,
    incidence_hiv_pos_high,
    case_detection_rate_percent,
```

```sql
    case_detection_rate_percent_low,
    case_detection_rate_percent_high
)
SELECT
    c.country_key,
    d.date_key,
    m.methodology_key,
    s.estimated_total_population,
    s.prevalence_per_100k,
    s.prevalence_per_100k_low,
    s.prevalence_per_100k_high,
    s.prevalence_total,
    s.prevalence_total_low,
    s.prevalence_total_high,
    s.mortality_excl_hiv_per_100k,
    s.mortality_excl_hiv_per_100k_low,
    s.mortality_excl_hiv_per_100k_high,
    s.deaths_excl_hiv,
    s.deaths_excl_hiv_low,
    s.deaths_excl_hiv_high,
    s.mortality_hiv_pos_per_100k,
    s.mortality_hiv_pos_per_100k_low,
    s.mortality_hiv_pos_per_100k_high,
    s.deaths_hiv_pos,
    s.deaths_hiv_pos_low,
    s.deaths_hiv_pos_high,
    s.incidence_per_100k,
    s.incidence_per_100k_low,
    s.incidence_per_100k_high,
    s.incident_cases,
    s.incident_cases_low,
    s.incident_cases_high,
    s.hiv_in_incident_tb_percent,
    s.hiv_in_incident_tb_percent_low,
    s.hiv_in_incident_tb_percent_high,
    s.incidence_hiv_pos_per_100k,
    s.incidence_hiv_pos_per_100k_low,
    s.incidence_hiv_pos_per_100k_high,
    s.incidence_hiv_pos,
    s.incidence_hiv_pos_low,
    s.incidence_hiv_pos_high,
    s.case_detection_rate_percent,
    s.case_detection_rate_percent_low,
    s.case_detection_rate_percent_high
FROM tb_reporting.stg_tb_burden_country_clean s
JOIN tb_reporting.dim_country c
    ON s.country_name = c.country_name
   AND COALESCE(s.iso3_code, '') = COALESCE(c.iso3_code, '')
JOIN tb_reporting.dim_date d
    ON s.year = d.year
JOIN tb_reporting.dim_methodology m
    ON COALESCE(s.prevalence_method, '') = COALESCE(m.prevalence_method, '')
   AND COALESCE(s.mortality_method, '') = COALESCE(m.mortality_method, '')
```

```
    AND COALESCE(s.incidence_method, '') = COALESCE(m.incidence_method, '')
    AND COALESCE(s.tbhiv_method, '') = COALESCE(m.tbhiv_method, '');
```

Performance Indexes

I add indexes on frequently joined and filtered columns so that reporting queries and dashboard refreshes run more efficiently.

```
CREATE INDEX idx_dim_country_name ON tb_reporting.dim_country(country_name);
```

```
CREATE INDEX idx_dim_country_region ON tb_reporting.dim_country(region);
```

```
CREATE INDEX idx_dim_country_iso3 ON tb_reporting.dim_country(iso3_code);
```

```
CREATE INDEX idx_fact_tb_country_key ON tb_reporting.fact_tb_burden(country_key);
```

```
CREATE INDEX idx_fact_tb_date_key ON tb_reporting.fact_tb_burden(date_key);
```

```
CREATE INDEX idx_fact_tb_methodology_key ON tb_reporting.fact_tb_burden(methodology_key);
```

```
CREATE INDEX idx_fact_tb_country_date ON tb_reporting.fact_tb_burden(country_key, date_key);
```

Reporting Views

To simplify downstream reporting, I create two reusable views: one detailed country-year reporting view and one aggregated region-year summary view.

```
CREATE VIEW tb_reporting.vw_tb_country_year_report AS
SELECT
    f.tb_burden_key,
    d.year,
    d.decade,
    c.country_name,
    c.iso2_code,
    c.iso3_code,
    c.iso_numeric_code,
    c.region,
    m.prevalence_method,
    m.mortality_method,
    m.incidence_method,
    m.tbhiv_method,
    f.estimated_total_population,
    f.prevalence_per_100k,
    f.prevalence_per_100k_low,
    f.prevalence_per_100k_high,
    f.prevalence_total,
    f.prevalence_total_low,
    f.prevalence_total_high,
    f.mortality_excl_hiv_per_100k,
```

```sql
    f.mortality_excl_hiv_per_100k_low,
    f.mortality_excl_hiv_per_100k_high,
    f.deaths_excl_hiv,
    f.deaths_excl_hiv_low,
    f.deaths_excl_hiv_high,
    f.mortality_hiv_pos_per_100k,
    f.mortality_hiv_pos_per_100k_low,
    f.mortality_hiv_pos_per_100k_high,
    f.deaths_hiv_pos,
    f.deaths_hiv_pos_low,
    f.deaths_hiv_pos_high,
    f.incidence_per_100k,
    f.incidence_per_100k_low,
    f.incidence_per_100k_high,
    f.incident_cases,
    f.incident_cases_low,
    f.incident_cases_high,
    f.hiv_in_incident_tb_percent,
    f.hiv_in_incident_tb_percent_low,
    f.hiv_in_incident_tb_percent_high,
    f.incidence_hiv_pos_per_100k,
    f.incidence_hiv_pos_per_100k_low,
    f.incidence_hiv_pos_per_100k_high,
    f.incidence_hiv_pos,
    f.incidence_hiv_pos_low,
    f.incidence_hiv_pos_high,
    f.case_detection_rate_percent,
    f.case_detection_rate_percent_low,
    f.case_detection_rate_percent_high
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c
    ON f.country_key = c.country_key
JOIN tb_reporting.dim_date d
    ON f.date_key = d.date_key
LEFT JOIN tb_reporting.dim_methodology m
    ON f.methodology_key = m.methodology_key;
```

```sql
CREATE VIEW tb_reporting.vw_tb_region_year_summary AS
SELECT
    d.year,
    c.region,
    COUNT(*) AS country_rows,
    SUM(f.estimated_total_population) AS total_population,
    AVG(f.prevalence_per_100k) AS avg_prevalence_per_100k,
    AVG(f.mortality_excl_hiv_per_100k) AS avg_mortality_excl_hiv_per_100k,
    AVG(f.incidence_per_100k) AS avg_incidence_per_100k,
    AVG(f.case_detection_rate_percent) AS avg_case_detection_rate_percent,
    SUM(f.incident_cases) AS total_incident_cases,
    SUM(f.deaths_excl_hiv) AS total_deaths_excl_hiv,
    SUM(f.deaths_hiv_pos) AS total_deaths_hiv_pos
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c
    ON f.country_key = c.country_key
```

```sql
JOIN tb_reporting.dim_date d
    ON f.date_key = d.date_key
GROUP BY d.year, c.region;
```

Validation Queries

These checks confirm that the raw, staging, dimension, and fact layers have loaded correctly and that the business grain remains unique.

```sql
SELECT COUNT(*) AS raw_count
FROM tb_reporting.raw_tb_burden_country;
```

Table 4: 1 records

raw_count
5120

```sql
SELECT COUNT(*) AS stage_count
FROM tb_reporting.stg_tb_burden_country_clean;
```

Table 5: 1 records

stage_count
5120

```sql
SELECT COUNT(*) AS fact_count
FROM tb_reporting.fact_tb_burden;
```

Table 6: 1 records

fact_count
5120

```sql
SELECT COUNT(*) AS country_count
FROM tb_reporting.dim_country;
```

Table 7: 1 records

country_count
219

```sql
SELECT COUNT(*) AS year_count
FROM tb_reporting.dim_date;
```

Table 8: 1 records

year_count
24

```
SELECT country_key, date_key, COUNT(*) AS duplicate_count
FROM tb_reporting.fact_tb_burden
GROUP BY country_key, date_key
HAVING COUNT(*) > 1;
```

Table 9: 0 records

country_key	date_key	duplicate_count

Sample Reporting Queries

Finally, I include a few analytical queries that can be used directly in reporting or dashboard prototypes.

```
SELECT
    c.country_name,
    d.year,
    f.incidence_per_100k,
    f.case_detection_rate_percent,
    f.deaths_excl_hiv
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c ON f.country_key = c.country_key
JOIN tb_reporting.dim_date d ON f.date_key = d.date_key
WHERE c.country_name = 'India'
ORDER BY d.year;
```

Table 10: Displaying records 1 - 10

country_name	year	incidence_per_100k	case_detection_rate_percent	deaths_excl_hiv
India	1990	217	81	330000
India	1991	216	81	340000
India	1992	216	57	350000
India	1993	216	54	350000
India	1994	216	55	360000
India	1995	216	59	370000
India	1996	217	61	370000
India	1997	217	53	380000
India	1998	217	50	390000
India	1999	217	55	400000

```
SELECT
    d.year,
    c.country_name,
    c.region,
```

```
    f.incident_cases,
    f.deaths_excl_hiv
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c ON f.country_key = c.country_key
JOIN tb_reporting.dim_date d ON f.date_key = d.date_key
WHERE d.year = 2013
ORDER BY f.incident_cases DESC NULLS LAST
LIMIT 10;
```

Table 11: Displaying records 1 - 10

year	country_name	region	incident_cases	deaths_excl_hiv
2013	India	SEA	2100000	240000
2013	China	WPR	980000	41000
2013	Nigeria	AFR	590000	160000
2013	Pakistan	EMR	500000	49000
2013	Indonesia	SEA	460000	64000
2013	South Africa	AFR	450000	25000
2013	Bangladesh	SEA	350000	80000
2013	Philippines	WPR	290000	27000
2013	Democratic Republic of the Congo	AFR	220000	46000
2013	Ethiopia	AFR	210000	30000

```
SELECT *
FROM tb_reporting.vw_tb_region_year_summary
ORDER BY year, region;
```

Table 12: Displaying records 1 - 10

year	region	country_count	total_population	prevalence_per_100k	avg_mortality_excl_hiv	avg_incidence_per_100k	avg_case_detection_pct	total_incident_cases	total_deaths_excl_hiv	deaths_hiv_pos
1990	AFR	46	502970338	317.13043	59.995652	249.63043	40.15349	1369057.0	351637.10	126277
1990	AMR	45	726564416	64.42889	5.784444	51.77111	64.23158	401466.5	38816.86	6018
1990	EMR	22	378032242	283.54545	22.324091	103.68182	67.97400	514550.0	118651.80	184
1990	EUR	53	848557038	84.18113	4.256793	50.21698	78.11765	364476.2	38929.81	531
1990	SEA	10	1309996655	500.30000	83.660000	271.90000	39.61111	2881520.0	646262.00	2083
1990	WPR	36	1531602630	307.64722	28.359722	156.40556	68.09000	2495012.2	332699.40	546
1991	AFR	46	517234627	707.10870	58.445652	253.82609	40.57500	1431053.0	358916.10	144596
1991	AMR	45	737714156	63.11111	5.660667	51.79778	65.18500	400363.8	37429.89	7041
1991	EMR	22	388179500	079.36364	21.298636	103.72727	73.33333	520770.0	120492.90	239
1991	EUR	53	852421129	91.73774	4.251887	53.68302	78.49583	395948.1	39551.42	657

Conclusion

This RMarkdown workflow transforms the raw TB burden CSV into a structured PostgreSQL star schema designed for reporting analysis. By separating descriptive dimensions from numeric facts and preserving a clean country-year grain, the model becomes easier to query, easier to connect to BI tools, and easier to maintain over time. It also creates a repeatable database build process that supports both technical documentation and production-style reporting workflows.